Huffman Dynamique

Algorithmique Avancée

UM4IN0500

Sofija GRANET 21208789
Thibaut MARCQ 21202966

Master Informatique - Semestre 1



Table des matières

Table des matières **2**

1 Introduction **3**
Question 1 . . . 3
Question 2 . . . 3

2 Implantation d'outils **4**
Question 3 . . . 4
Question 4 . . . 4

3 Compression et Décompression **5**
Question 5 . . . 5
Question 6 . . . 5
Question 7 . . . 6

4 Étude expérimentale **7**
Question 8 . . . 7
Question 9 . . . 8
Question 10 . . . 9
Question 11 . . . 10

5 Compléments **12**
Comment lancer le programme ? . . . 12
Utilisation des outils d'intelligence artificielle [ok pour relecture] . . . 13
Justification des choix technologiques[ok pour relecture] . . . 13
Architecture du projet [ok pour relecture] . . . 14

1 Introduction

Question 1 Montrer que l'algorithme de modification de l'arbre n'échange jamais un nœud avec un de ses ancêtres.

L'échange de nœuds dans l'arbre est fait dans Traitement *(cf diapo 48 du cours)*. On n'y échange que les sous-arbres enracinés en x_m et x_b, respectivement le premier et le dernier nœud de Γ_Q tels que $poids(x_m) = poids(x_{m+1})$. Ainsi, on a

$$poids(x_m) = poids(x_b) \leq poids(x_{b+1})$$

Montrons que l'arbre enraciné en x_b n'est pas un ancêtre de x_m.
Supposons par l'absurde qu'il existe un arbre tel que x_m est dans un sous-arbre de x_b. Voyons ce que cela implique.

Rappelons que le parcours $GDBH$ numérote d'abord les sous-arbres gauche et droit d'un arbre avant sa racine, ce qui veut dire que tout descendant est visité **avant** son ancêtre. Sachant cela, on en déduit que x_b se trouve **avant** x_m dans ce parcours. De plus, on a obtenu x_b avec la fonction finBloc, qui renvoie le **dernier** nœud du bloc de x_m, c'est-à-dire que x_b apparaît **après** x_m dans le parcours GDBH **et** que $poids(x_m) = poids(x_b)$. On a donc bien

$$GDBH(x_m) < GDBH(x_b)$$

En revanche, x_b étant un ancêtre implique qu'il y a des nœuds internes qui le séparent de x_m ; or, dans un arbre de Huffman les poids augmentent lorsqu'on monte, i.e qu'on passe d'un niveau à un autre de l'arbre, d'où :

$$poids(x_m) < poids(x_b)$$

On a donc une contradiction ; les poids des deux nœuds sont à la fois égaux **et** strictement différents. On en conclut que l'arbre enraciné en x_b n'est pas un ancêtre de x_m et donc que l'algorithme de modification de l'arbre n'échange jamais un nœud avec un de ses ancêtres.

Question 2 Quel est le nombre maximal d'échanges réalisés lors d'une modification de l'arbre (en fonction de la taille de l'alphabet des symboles) ?

Dans le pire cas, **tous** les nœuds du chemin vers la racine ne sont pas incrémentables, donc il faut effectuer un échange à chaque niveau de l'arbre. Or, le chemin d'une feuille jusqu'à la racine a une longueur au plus égale à la hauteur h de l'arbre. On a donc un nombre maximal d'échanges du même ordre que la hauteur de l'arbre, d'où $O(h)$. Par ailleurs, la hauteur de l'arbre dépend du nombre de symboles dans l'alphabet. Selon la distribution des symboles dans le texte, l'arbre peut être plus ou moins équilibré ; théoriquement dans le pire cas sa hauteur est de $h = n$, avec n la taille de l'alphabet, et dans le meilleur cas elle est de $h = log_2(n)$. En pratique on n'arrive pas réellement à ce pire cas, puisqu'on met à jour l'arbre à chaque insertion d'un nouveau symbole.

Expérimentalement, nous avons essayé d'évaluer le nombre maximal d'échanges sur plusieurs textes ; la valeur la plus grande que nous avons observée est de 3 échanges.

2 Implantation d'outils

Question 3 Écrire une fonction lecture prenant en entrée un fichier .bin et permettant la lecture du contenu du fichier et son affichage à l'écran sous forme d'une chaîne de bits.

```
def lecture(file : str) -> bitarray:
    """Lit un fichier UTF-8 et renvoie un bitarray"""
    bits = bitarray()
    with open(file, 'rb') as f:
        bits.fromfile(f)

    return bits
```

cf fichier src/utils.py ligne 267

Question 4 Écrire une fonction ecriture prenant en entrée fichier_chaine.txt et fichier.bin et produisant en sortie un fichier binaire nommé fichier.bin encodant le texte fichier_chaine.txt sous forme de fichier binaire.

```
def ecriture(fichier_chaine : str, fichier : str) -> str:
    """Encode le texte fichier_chaine sous forme de fichier binaire"""
    bin = lecture(fichier_chaine)
    with open('fichier.txt', 'r', encoding='utf-8') as f:
        text = f.read()
        bin.frombytes(text.encode('utf-8'))
        bin.tofile(f)

    return fichier
```

cf fichier src/utils.py ligne 317

Pour des raisons d'efficacité et de simplicité, on utilise la fonction writeBin ci-dessous dans le code de compression, à la place de la fonction ecriture.

```
def writeBin(bin : bitarray, file : str):
    """Ecrit un bitarray dans un fichier binaire"""
    with open(file, 'wb') as f:
        bin.tofile(f)
```

cf fichier src/utils.py ligne 312

3 Compression et Décompression

Question 5 Définir les structures de données nécessaires à l'implantation des algorithmes de compression et décompression. Rappeler la complexité des opérations sur ces structures.

Pour les algorithmes de compression et de décompression, on utilise une structure d'arbre binaire de recherche. Initialement, le seul nœud présent dans l'arbre est le caractère spécial ##. On utilise ce double caractère pour éviter tout problème lors de la lecture d'un caractère # (la lecture s'effectue caractère par caractère, il n'est pas possible d'insérer deux caractères à la fois).
Complexités des opérations sur l'arbre de Huffman :

- $createTree$: générer rapidement un arbre binaire, utilisé pour faciliter les tests sur nos fonctions $\rightarrow O(l)$, où l est la longueur de l'arbre souhaité
- $isLeaf$: test booléen pour évaluer si le nœud est une feuille $\rightarrow O(1)$
- $isNode$: test booléen pour évaluer si le nœud est un noeud interne $\rightarrow O(1)$
- $isRoot$: test booléen pour évaluer si le nœud est la racine de l'arbre $\rightarrow O(1)$
- $newLeft$ et $newRight$: créer un nouveau fils gauche ou droit d'un noeud donné $\rightarrow O(1)$
- $searchNode$: chercher un noeud en parcourant l'arbre $\rightarrow O(N)$ avec N le nombre de nœuds dans l'arbre
- $swapNodes$: échange deux nœuds dans l'arbre $\rightarrow O(1)$
- $getCode \rightarrow O(n)$ avec n le nombre de symboles dans l'alphabet, d'après les règles de construction de l'arbre de Huffman on n'arrive pas à ce cas (arbre peigne)

Question 6 Implanter la structure arborescente permettant de manipuler l'arbre de Huffman dynamique, ainsi que la fonction finBloc présentée dans le cours.

Voici ci-dessous notre implantation en Python de la structure d'arbre ; on stocke pour chaque nœud sa valeur et son poids, et on maintient des références vers ses fils et son parent.

```
def __init__(self, value: T, weight : int = 1):
    '''
    initial constructor
    '''
    self.value : T = value
    self.weight : int = weight
    self.left : Node = None
    self.right : Node = None
    self.parent : Node = None
```

cf fichier `src/Node.py` ligne 6

```python
def finBloc(self, root : "Node") -> "Node":
        """_Renvoie le dernier noeud du bloc de self dans le parcours GDBH
            => le dernier noeud qui a le même poids que self._"""
        b : Node = self
        gdbh : List["Node"] = self.GDBH(root)

        for node in gdbh :
            if node.weight == self.weight :
                b = node
            else :
                break

        return b
```

cf fichier `src/Node.py` ligne 216

Question 7 Tester les programmes de compression et décompression sur différents jeux de tests.

Les fonctions implémentant les algorithmes de compression et de décompression sont nommées respectivement `encode` et `decode`. Elles sont d'abord testées sur de petites instances dans `tests/tests.ipynb`.

Par exemple, la compression du texte `"carambarbcm"` produit le résultat suivant :
`01100011001100001000111001011100011011011100011000100111100110011001000`
Ce résultat est conforme au comportement attendu pour l'algorithme de Huffman dynamique. La décompression de cet encodage restitue correctement le texte original : `"carambarbcm"`.

Des tests supplémentaires ont ensuite été menés sur des textes plus longs, des livres entiers et divers fichiers. Les résultats détaillés sont présentés dans les questions suivantes.

Note : Toutes nos fonctions qui implémentent les opérations élémentaires sur l'arbre (*cf fichier src/utils.py*), dont le traitement, sont testées dans *tests/tests.ipynb*.

4 Étude expérimentale

Question 8 En vous basant sur le projet Gutenberg, par exemple, ou d'autres bibliothèques de textes, recueillir des résultats expérimentaux de la compression et de la décompression par la méthode de Huffman dynamique. Indiquer des références vers les fichiers utilisés.

cf fichier `tests/analyse.ipynb`

Nous avons utilisé les références suivantes (Projet Gutenberg) :

— Notes de Voltaire et de Condorcet sur les pensées de Pascal

— Notre Dame de Paris – Victor Hugo

— Frankenstein ; Or, The Modern Prometheus – Mary Wollstonecraft Shelley (anglais)

— Ιστορία της Αρχαίας Ελληνικής Λογοτεχνίας – Gilbert Murray (grec)

— 1001 задача для умственного счета – Sergei Aleksandrovich Rachinskii (russe)

— 李太白集 – Bai Li (chinois)

Voici les résultats obtenus :

Langue (Auteur)	**Nb. symboles**	**Taille (octets)**	**Taux**	**Facteur**	**Temps (s)**
Français (Hugo)	117	289 580 / 528 986	45,26%	1,83x	15,42
Blaise Pascal	109	66 246 / 120 304	44,93%	1,82x	3,30
Anglais	93	244 492 / 448 929	45,54%	1,84x	11,48
Grec	179	589 106 / 1 561 597	62,28%	2,65x	43,19
Russe	150	112 908 / 271 490	58,41%	2,40x	7,41
Chinois	4 408	278 776 / 623 952	55,32%	2,24x	333,64

On peut dégager deux analyses de ces résultats : le temps d'exécution de l'algorithme de compression varie en fonction de la taille des textes, ainsi qu'en fonction du nombre de symboles/caractères dans le texte.

On remarque que les deux textes en français (plage de symboles UTF-8 française classique en commun) ont un temps de compression qui varie en fonction de leur taille. Plus un texte est grand, plus on met de temps à le compresser.
Cela est simplement lié au nombre d'opérations à traiter au sein de l'arbre.

On peut aussi remarquer que plus un texte contient un alphabet grand (texte chinois), plus l'algorithme de compression met du temps à s'exécuter. Le texte chinois est moins grand que le texte grec en taille, mais à cause de sa grande diversité de caractères, son temps de compression augmente fortement.
Cela est dû à l'augmentation de la taille de l'arbre de Huffman. Sa taille dépend de la taille de l'alphabet en entrée. Plus le nombre de symboles différents est grand, plus l'arbre l'est aussi et plus il est ainsi long de le parcourir à chaque étape de l'algorithme.

Question 9 Reproduire ces expérimentations en utilisant des fichiers aléatoires par exemple (des fichiers textuels où les caractères sont générés aléatoirement). Expliquer comment les fichiers ont été générés.

cf fichier `tests/analyse.ipynb`

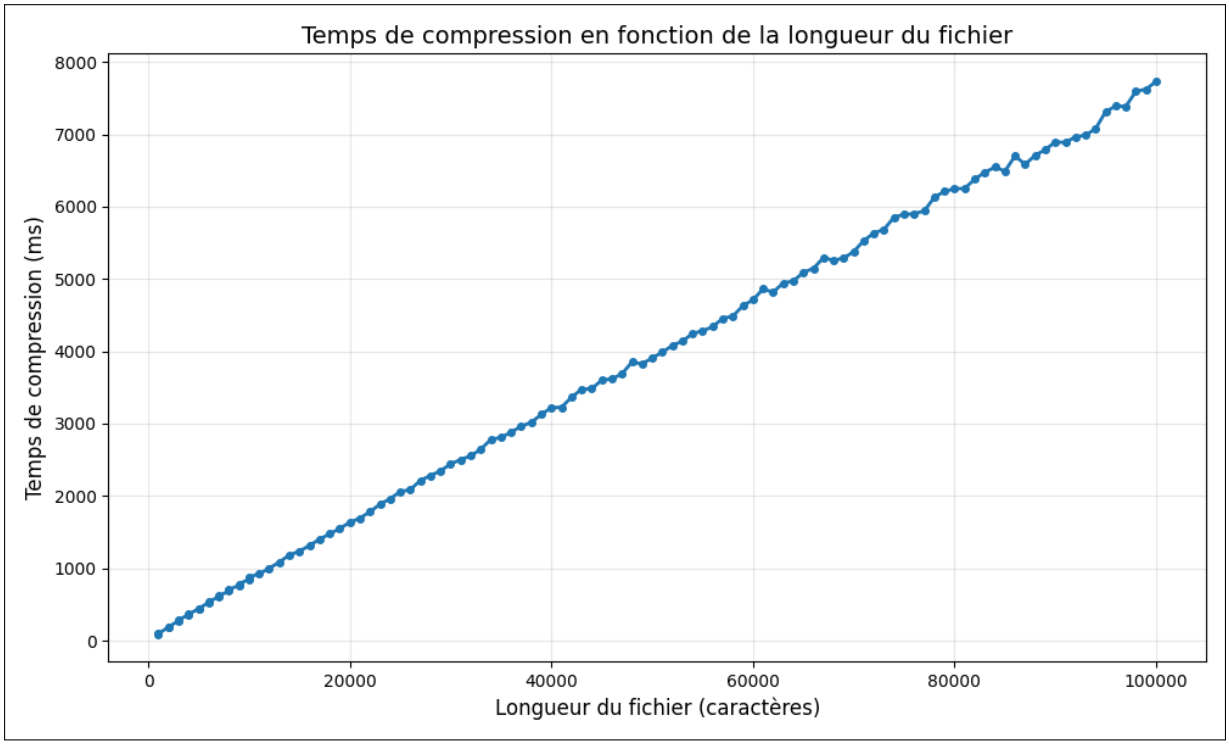
Comment génère-t-on les fichiers textuels pour ces expérimentations ?
Pour créer des fichiers textuels où les caractères sont générés aléatoirement, on lance le script generer.sh comme ci-dessous, à partir de la racine du projet :

```
./tests/generer.sh <nombre_fichiers>
```

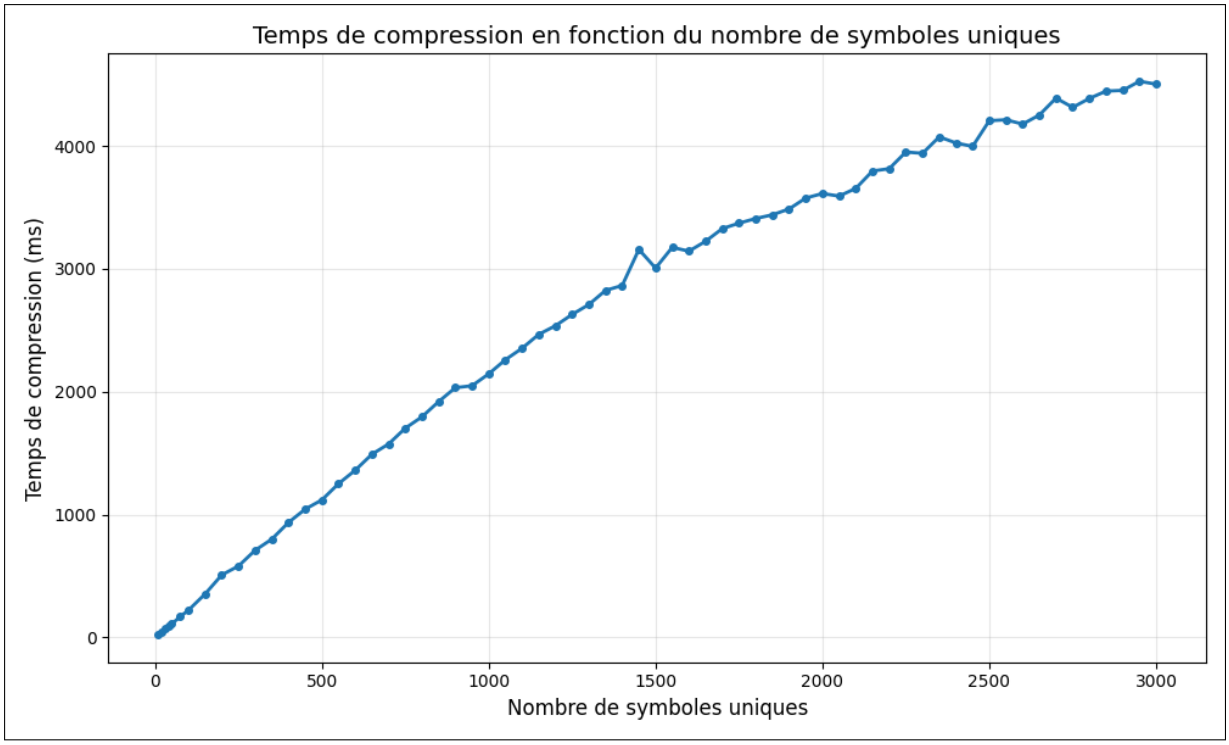
Plus de détails sur ce script sont données à la question 11, dans laquelle on l'utilise pour étudier la répartition des caractères dans nos textes.

Expérimentations
Dans un premier temps, on génère des fichiers de tailles différentes avec une taille d'alphabet fixée. Dans un deuxième temps on génère ensuite des fichiers avec des tailles d'alphabet différentes, en fixant la longueur du texte.

Voici les résultats obtenus :



Génération seulement à partir de caractères latins



Fichiers générés avec une taille fixe = 10000 caractères

Question 10 Reproduire ces expérimentations en utilisant des fichiers qui contiennent de l'information mais pas directement sous forme de texte en langue naturelle (par exemple des fichiers .json, .py, .c, . . .).

cf fichier tests/analyse.ipynb

Nous avons utilisé des fichiers qui contiennent du code (i.e pas en langage naturel). On utilise ici des fichiers au format `.py`, `.ipynb`, `.md` et `.sh` : on se servira tout simplement de fichiers de code présents dans ce projet.

Voici les résultats obtenus :

Nom, format	Nb. symboles	Taille (octets)	Taux	Facteur	Temps (s)
generate.py	322	5663 / 4013	70,86%	1.41x	3,96
Node.py	73	9006 / 4942	54,89%	1,82x	2,13
utils.py	87	11810 / 6823	57,77%	1,73x	2,96
tests.ipynb	87	17334 / 10399	59,99%	1,67x	4,45
README.md	82	1373 / 1023	74,51%	1,67x	0,47
generer.sh	89	3543 / 2331	65,79%	1,52x	1,10

Ces résultats concordent avec ceux observés sur les fichiers textuels tirés du projet Gutenberg, vus plus haut.

Question 11 À l'aide des résultats recueillis lors des dernières questions, proposer une analyse expérimentale de la compression et la décompression par la méthode de Huffman dynamique.

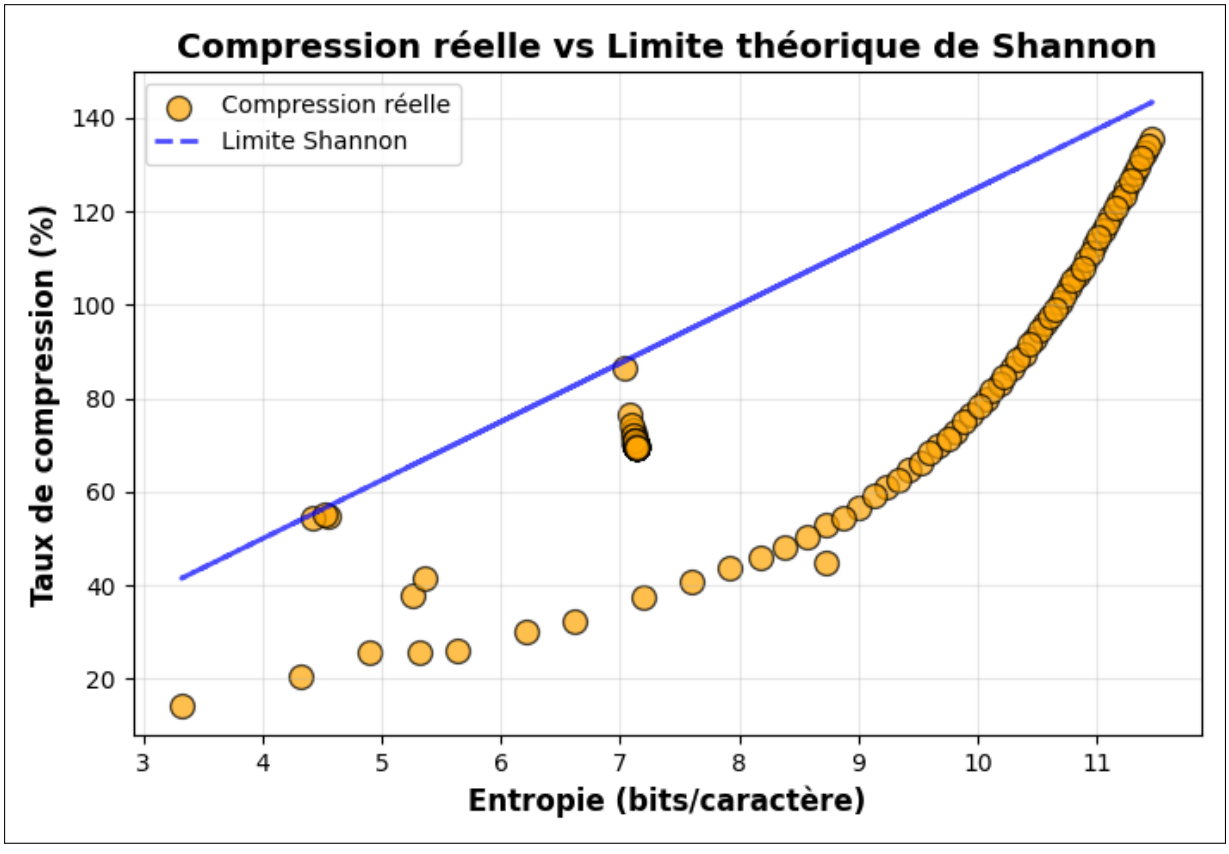
D'après les résultats expérimentaux des questions précédentes, nous pouvons tirer plusieurs conclusions.

— Le temps de compression dépend de la taille du texte à compresser (i.e le nombre de caractères). On observe que la courbe suit une tendance linéaire.

— Le temps de compression dépend aussi de la taille de l'alphabet des symboles (i.e le nombre de caractères différents). On observe une courbe à tendance logarithmique. La forme de cette courbe peut s'expliquer par la variation de la hauteur de l'arbre de Huffman ; à chaque nouveau symbole inséré, celle-ci augmente de façon logarithmique.

Enfin, on peut analyser le taux de compression de nos textes pour tirer d'autres conclusions sur nos données.

On réutilise le script utilisé à la question 9. En plus de générer des fichiers de différentes longueurs comme présenté précédemment, ce script permet aussi de générer des fichiers selon une distribution de caractères. On ajoute à ces fichiers les fichiers en langage naturel pour obtenir un plage d'entropies assez larges et on observe comment se comporte le taux de compression de tous ces fichiers.



Taux de compression en fonction de l'entropie du texte

Sur ce graphique on peut voir que l'entropie du texte influence directement le taux de compression. Cela signifie que plus la répartition des fréquences des caractères dans le texte est inégale, plus le taux de compression sera élevé (donc meilleur).

C'est une conclusion cohérente. Un texte dans lequel les fréquences d'apparition des caractères sont variées va signifier qu'il est possible d'attribuer un petit code aux caractères avec une fréquence élevé, tandis que l'on va attribuer un code plus long aux caractères dont la fréquence est faible. C'est de cette façon que l'on gagne de la place.

Le script de génération aléatoire fonctionne de la façon suivante ; il exécute generate.py, qui génère une chaîne de caractères grâce aux bibliothèques string et random, et qui la sauvegarde dans un fichier texte. Il génère du texte aléatoire, multilingue, et avec emojis :

- latin avec lettres et diacritiques divers ajoutés (français, lituanien, espagnol et norvégien)
- chinois simplifié (caractères courants)
- japonais (Hiragana et Katakana)
- coréen
- cyrilique
- grec
- quelques emojis

On peut personnaliser la distribution de probabilité de chaque alphabet dans le texte généré. Le script compresse ensuite les fichiers générés avec le script de compression [1].

1. *cf section 5*

5 Compléments

Comment lancer le programme ?

Le code de compression s'exécute avec un script bash qui se lance comme décrit ci-dessous à partir de la racine du projet :

```
./compresser nom_fichier.txt nom_fichier_compresse.huff
```

À chaque exécution de cette commande, un fichier compression.txt est mis-à-jour en ajoutant une ligne contenant :

1. le nom du fichier d'entrée (texte original .txt)
2. le nom du fichier de sortie (fichier compressé .huff)
3. le nombre d'octets du fichier d'entrée
4. le nombre d'octets du fichier de sortie
5. le taux de compression (nombre d'octets du fichier compressé divisé par le nombre d'octets du fichier original), en ne conservant que 5 décimales
6. le temps de compression en millisecondes

Chaque information est séparée de la suivante par un point virgule.

De façon analogue, le code de décompression s'exécute avec un script bash qui se lance comme décrit ci-dessous à partir de la racine du projet :

```
./decompresser nom_fichier_compresse.huff nom_fichier.txt
```

À chaque exécution de cette commande, un fichier decompression.txt est mis-à-jour en ajoutant une ligne contenant :

1. le nom du fichier d'entrée (fichier compressé .huff)
2. le nom du fichier de sortie (texte original .txt)
3. le nombre d'octets du fichier d'entrée
4. le nombre d'octets du fichier de sortie
5. le taux de compression (nombre d'octets du fichier compressé divisé par le nombre d'octets du fichier original), en ne conservant que 5 décimales
6. le temps de compression en millisecondes

Chaque information est séparée de la suivante par un point virgule.

Utilisation des outils d'intelligence artificielle

Pour nous aider dans notre code, nous avons eu recours à des assistants d'intelligence artificielle intégrés à l'extension *Copilot* dans *Visual Studio Code*. Les modèles principalement utilisés étaient **GPT-5** et **Claude Sonnet 4.5**.

Nous avons utilisé l'IA uniquement de manière **complémentaire** à ce que nous avions déjà codé. Leur utilisation s'est concentrée sur :

- la **relecture critique** de notre code et la **correction d'erreurs** que nous n'arrivions pas à résoudre, souvent liées à des problèmes d'inattention ;
- la **génération d'un mémo** en rapport avec le module Python `bitarray`. Cela nous a permis d'avoir une vue globale du module, ainsi que des méthodes à utiliser. Cela nous a fait gagner un temps considérable ;
- la **génération de fichiers de tests** pour des instances variées, ainsi qu'une aide dans la création de scripts `Bash` en fin de projet.

Estimation quantitative

Nous estimons que **90% du code** a été écrit manuellement, tandis que **10%** a été généré par IA.

Utilisation pour le rapport

L'apport de l'IA dans la rédaction du rapport a été **minime**. Elle n'a pas été utilisée pour rédiger le contenu, mais uniquement pour améliorer la **lisibilité** de certaines sections et ajouter des **éléments visuels** plus agréables.

Justification des choix technologiques

Nous avons décidé d'implémenter ce projet en **Python** pour plusieurs raisons :

- **Simplicité de la syntaxe** : Python facilite l'implémentation d'algorithmes complexes, comme celui de Huffman dynamique.
- **Lisibilité** : la clarté du code permet une maintenance aisée et favorise une collaboration efficace.
- **Gestion automatique de la mémoire** : grâce à son *garbage collector*, Python évite la gestion manuelle de la mémoire, contrairement au C/C++ par exemple, ce qui est particulièrement pratique pour un projet de cette envergure.
- **Bibliothèques** : Python propose de nombreuses bibliothèques utiles, notamment pour la gestion des bits (`bitarray`) et la visualisation de graphes (`pandas`, `matplotlib`...).

Architecture du projet

Le projet est structuré dans différents sous-répertoires afin de garantir une meilleure lisibilité et de faciliter l'écriture du code.

Arborescence générale

```
Huffman-Dynamique/
|   src/
|   |   generate.py        (génération de textes aléatoires)
|   |   Node.py            (classe représentant l'arbre de Huffman)
|   |   utils.py
|
|   tests/
|   |   analyse.ipynb
|   |   generer_analyse_len.sh
|   |   generer_analyse_symboles.sh
|   |   generer.sh         (script de génération aléatoire)
|   |   tests.ipynb        (tests de fonctions diverses)
|
|   texts/
|   |   analyse/
|   |   |   len/           (textes aléatoires, variation longueur texte)
|   |   |   symboles/      (textes aléatoires, variation taille alphabet)
|   |   random/            (textes aléatoires, alphabets variés)
|   |   bintexts/          (textes compressés .huff)
|   |   (textes en langage naturel .txt)
|
|   compression.txt        (fichier log de la compression)
|   decompression.txt      (fichier log de la décompression)
|   compresser             (script de lancement de la compression)
|   decompresser           (script de lancement de la décompression .huff)
|   encode.py
|   decode.py
```